

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ «МИСиС»

ИНСТИТУТ _____ Информационных компьютерных наук _____

КАФЕДРА _____ Инфокоммуникационных технологий _____

НАПРАВЛЕНИЕ _____ 09.03.01 _____ ГРУППА БИВТ 24-5

КУРСОВОЙ ПРОЕКТ

ПО КУРСУ Клиент-серверные приложения и сетевые технологии.

ТЕМА Район

СТУДЕНТ Рочев Е.В.

РУКОВОДИТЕЛЬ ПРОЕКТА _____ Микитенко И.И., Абросимов Н.А.,

Шелудяков П.А..



ОЦЕНКА _____

Москва, 2026

СОДЕРЖАНИЕ

1. Предметная область	3
2. Постановка задачи	3
3. Описание архитектуры.....	4
4. Описание структуры БД	5
4.1. Вербальная модель	5
4.2. Схема базы данных.....	5
4.3. Объекты базы данных	7
5. Описание серверной части	8
6. Описание клиентской части	12
7. Заключение.....	15
Список использованных источников.....	17

1. Предметная область

Городской район является базовой административно-территориальной единицей крупного города. Каждый район обладает уникальной инфраструктурой, включающей объекты здравоохранения, образования, культуры, спорта и торговли, а также регулярно проводимыми общественными мероприятиями. Управление актуальными сведениями о районах представляет значительную практическую ценность как для городской администрации, так и для жителей и туристов.

Основная проблема в данной предметной области заключается в отсутствии единого структурированного инструмента для хранения и оперативного управления информацией о районах города, объектах их инфраструктуры и проводимых мероприятиях. Существующие решения, как правило, представлены разрозненными таблицами или статичными веб-страницами, не допускающими гибкого управления данными в рамках единой системы.

Разрабатываемое клиент-серверное приложение направлено на решение задачи структурированного хранения и управления информацией о городских районах, включая характеристики районов, типы и объекты инфраструктуры, а также проводимые мероприятия. Дополнительно система реализует разграничение прав доступа между категориями пользователей: администраторами (admin) и наблюдателями (viewer).

2. Постановка задачи

Вариант 21. Тема: Район.

Целью данной курсовой работы является разработка полнофункционального клиент-серверного веб-приложения для управления базой данных городских районов. Приложение должно обеспечивать хранение, отображение и редактирование информации о районах, объектах инфраструктуры, типах инфраструктуры и событиях, а также предоставлять систему авторизации с разграничением прав доступа.

Функциональные возможности разрабатываемого приложения:

- регистрация и авторизация пользователей с использованием JWT-токенов и хешированием паролей через bcryptjs (10 раундов соли);
- управление доступом по категориям пользователей: роль «admin» - полный CRUD, роль «viewer» - только чтение;
- хранение JWT-токена и данных пользователя в LocalStorage браузера с поддержкой сессии между перезагрузками;
- просмотр полного списка районов с фильтрацией по названию и описанию;

- добавление, редактирование и удаление записей (CRUD) для районов, объектов инфраструктуры, типов инфраструктуры и событий;
- управление объектами инфраструктуры с указанием адреса, года постройки, района и типа объекта;
- управление событиями в районах с указанием даты и описания;
- отображение связанной информации: количество объектов и событий в районе;
- логирование всех входящих HTTP-запросов через NestJS Middleware (метод, URL, IP, временная метка);
- реализация модуля Users с хранением данных в памяти (in-memory) в соответствии с требованиями задания: эндпоинты GET /users, GET /users/:id, POST /users с обязательными полями id, name, email и необязательным age, валидацией email;
- защита маршрутов на клиенте через компонент ProtectedRoute с перенаправлением неаутентифицированных пользователей на страницу входа.

3. Описание архитектуры

Разработанное приложение реализует трёхуровневую клиент-серверную архитектуру, в которой выделяются следующие уровни: уровень представления (клиентская часть), уровень бизнес-логики (серверная часть) и уровень данных (база данных).

Взаимодействие между клиентом и сервером осуществляется посредством REST API. Клиентская часть отправляет HTTP-запросы (GET, POST, PUT, DELETE) на сервер, который выполняет бизнес-логику и обращается к базе данных через ORM TypeORM. Для корректной работы в режиме разработки в Vite настроен прокси-сервер, перенаправляющий запросы с порта 5173 на порт 3000. Аутентификация реализована на основе JWT-токенов (срок действия 1 день): при входе сервер возвращает токен, который клиент сохраняет в LocalStorage и передаёт в заголовке Authorization при каждом защищённом запросе.

В таблице 1 приведено описание основных технологий и их назначения в рамках разработанной архитектуры.

Таблица 1 - Технологический стек приложения

Компонент	Технология	Назначение
Клиентская часть	React 18 + Vite + TypeScript	Пользовательский интерфейс, маршрутизация, отображение данных
Серверная часть	NestJS 10 + TypeScript	REST API, бизнес-логика, аутентификация, взаимодействие с БД

Компонент	Технология	Назначение
База данных	PostgreSQL 16	Хранение реляционных данных
ORM	TypeORM	Объектно-реляционное отображение сущностей, репозитории
HTTP-клиент	Axios	HTTP-запросы с клиента к серверу, обработка ответов
Безопасность	@nestjs/jwt + bcryptjs	JWT-аутентификация, хеширование паролей (bcrypt, 10 раундов)
Валидация	class-validator + class-transformer	Проверка и преобразование входных данных на уровне DTO
Управление сессией	LocalStorage API	Хранение JWT-токена и данных пользователя на клиенте

4. Описание структуры БД

4.1. Вербальная модель

База данных приложения содержит пять таблиц, связанных между собой отношениями вида «один ко многим».

Сущность «Район» (district) хранит информацию о городском районе: название (уникальное), площадь в квадратных километрах, численность населения, год основания и текстовое описание. Один район может включать множество объектов инфраструктуры и событий.

Сущность «Тип инфраструктуры» (infrastructure_type) описывает категорию объекта (школа, больница, парк и т.д.) и содержит его уникальное название и описание. Один тип может быть присвоен нескольким объектам инфраструктуры.

Сущность «Объект инфраструктуры» (infrastructure_object) содержит название объекта, адрес, год постройки, а также ссылки на район (district_id) и тип инфраструктуры (type_id). При удалении района объекты удаляются каскадно; при удалении типа поле type_id обнуляется (SET NULL).

Сущность «Событие» (event) хранит название мероприятия, его дату, описание и ссылку на район проведения (district_id). При удалении района события удаляются каскадно.

Сущность «Журнал аудита» (district_audit_log) является технической таблицей, заполняемой автоматически триггером при операциях INSERT и DELETE над таблицей district. Хранит идентификатор района, тип операции и временную метку.

4.2. Схема базы данных

В таблице 2 представлено описание структуры таблиц базы данных приложения.

Таблица 2 - Структура таблиц базы данных

Таблица	Поле	Тип	Описание
district	id	SERIAL PK	Идентификатор района
district	name	VARCHAR(100) UNIQUE	Название района
district	area_km2	NUMERIC(10,2)	Площадь (кв. км)
district	population	INT	Численность населения
district	foundation_year	INT	Год основания
district	description	TEXT	Описание района
infrastructure_type	id	SERIAL PK	Идентификатор типа
infrastructure_type	name	VARCHAR(100) UNIQUE	Название типа
infrastructure_type	description	TEXT	Описание типа
infrastructure_object	id	SERIAL PK	Идентификатор объекта
infrastructure_object	name	VARCHAR(200)	Название объекта
infrastructure_object	address	VARCHAR(255)	Адрес объекта
infrastructure_object	foundation_year	INT	Год постройки
infrastructure_object	district_id	INT FK	Ссылка на район
infrastructure_object	type_id	INT FK	Ссылка на тип инфраструктуры
event	id	SERIAL PK	Идентификатор события
event	name	VARCHAR(200)	Название события
event	event_date	DATE	Дата проведения
event	description	TEXT	Описание события
event	district_id	INT FK	Ссылка на район
district_audit_log	id	SERIAL PK	Идентификатор записи
district_audit_log	district_id	INT	ID района
district_audit_log	action	VARCHAR(20)	Тип операции: INSERT / DELETE

Таблица	Поле	Тип	Описание
district_audit_log	action_date	TIMESTAMP	Дата и время операции

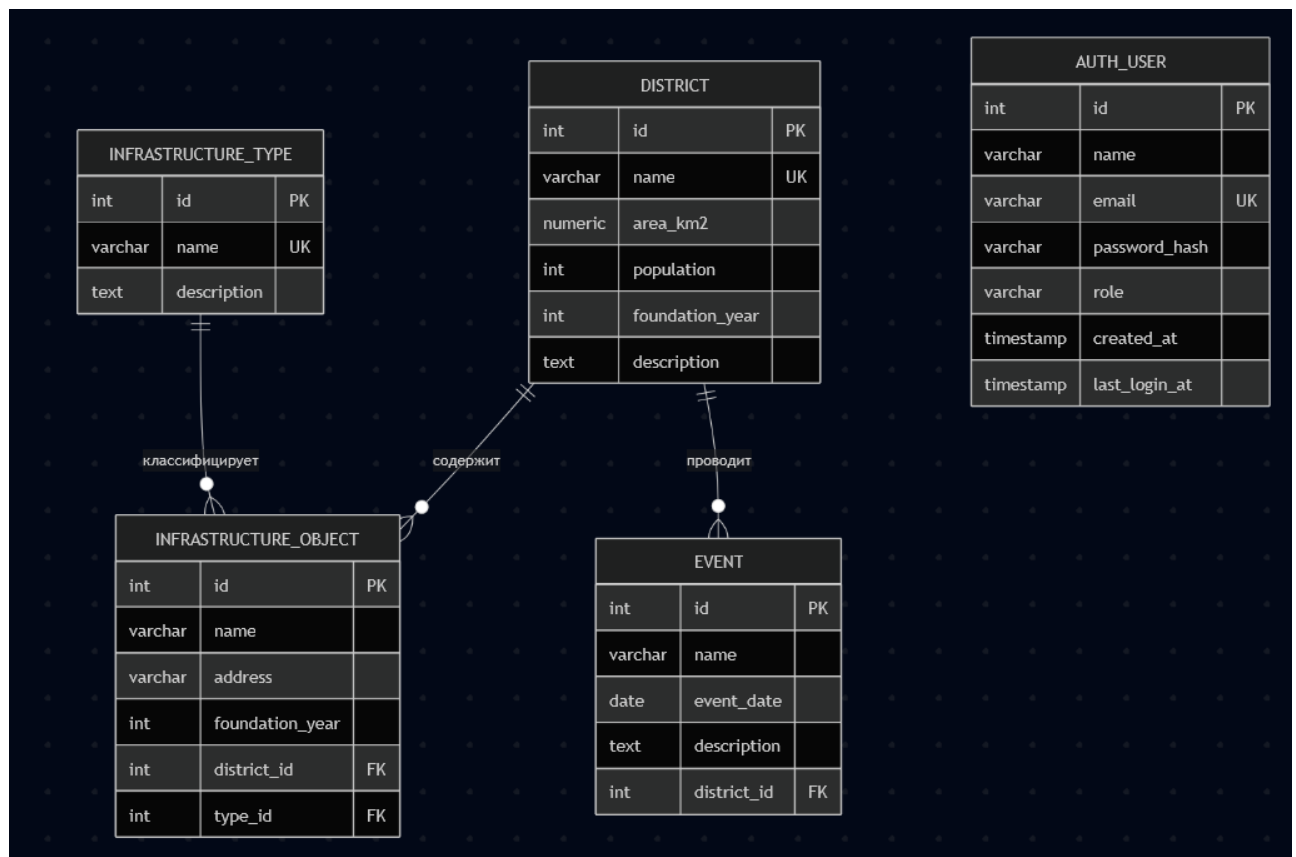


Рисунок 1 - ER-диаграмма базы данных приложения

4.3. Объекты базы данных

В базе данных реализованы представления (VIEW) [1]:

- **view_district_stats** - статистика по районам: численность населения, количество связанных объектов инфраструктуры (object_count) и событий (event_count), вычисляемые через LEFT JOIN с группировкой;
- **view_infrastructure_full** - полная информация об объектах инфраструктуры с наименованием района и типа объекта, получаемая через INNER JOIN таблиц infrastructure_object, district и infrastructure_type;
- **view_events_upcoming** - предстоящие события (event_date >= CURRENT_DATE) с наименованием района, упорядоченные по дате возрастания.

Функции, реализованные в базе данных:

- **get_objects_by_district(p_district_id INT)** - возвращает таблицу с названием объекта, адресом и типом инфраструктуры для указанного района;

- `get_district_population_rank()` - возвращает рейтинг всех районов по убыванию численности населения с использованием оконной функции `RANK() OVER`;
- `get_events_by_district(p_district_id INT)` - возвращает список событий указанного района, упорядоченный по дате.

Хранимые процедуры:

- `add_district_with_default_events` - добавляет новый район и автоматически создаёт для него два события-заглушки («День нового района» через 30 дней и «Общественные слушания» через 45 дней), выполняется в одной транзакции;
- `delete_district_cascade` - явно удаляет все связанные события и объекты инфраструктуры, затем сам район, демонстрируя логику транзакционного удаления;
- `transfer_objects` - перемещает все объекты инфраструктуры из одного района в другой обновлением поля `district_id`.

Триггеры:

- `trg_check_population_positive` (BEFORE INSERT OR UPDATE ON district) - запрещает сохранение записи с отрицательным значением населения, вызывая исключение;
- `trg_district_audit` (AFTER INSERT OR DELETE ON district) - автоматически записывает в таблицу `district_audit_log` идентификатор района и тип выполненной операции;
- `trg_check_event_date` (BEFORE INSERT OR UPDATE ON event) - запрещает создание события с датой в прошлом, вызывая исключение.

5. Описание серверной части

Серверная часть приложения реализована на фреймворке NestJS [2] и организована по модульному принципу. Каждый модуль отвечает за работу с одной сущностью и включает четыре компонента: `entity`, `dto`, `service` и `controller`.

`Entity` - класс, описывающий структуру таблицы базы данных с помощью декораторов `TypeORM` (`@Entity`, `@PrimaryGeneratedColumn`, `@Column`, `@ManyToOne`, `@OneToMany` и др.).

`DTO` (`Data Transfer Object`) - классы для описания структуры входящих данных. Используют декораторы `class-validator`: `@IsNotEmpty`, `@IsEmail`, `@IsInt`, `@Min`, `@IsOptional`, `@IsDateString`.

`Service` - содержит бизнес-логику: методы `findAll()`, `findOne()`, `create()`, `update()`, `remove()`. Взаимодействует с базой данных через `Repository` из `TypeORM` [4]. При отсутствии записи выбрасывает `NotFoundException`.

Controller - обрабатывает HTTP-запросы и делегирует операции сервису. Использует декораторы @Get(), @Post(), @Put(), @Delete(), @Param(), @Body(), @ParseIntPipe.

Дополнительно реализованы: JwtAuthGuard - Guard для защиты маршрутов, требующих аутентификации; RolesGuard - проверка роли пользователя; LoggingMiddleware - логирование всех входящих запросов с записью метода, URL, IP и временной метки.

В таблице 3 представлены модули серверной части приложения.

Таблица 3 - Модули серверной части

Модуль	Controller	Service	Маршруты API
AuthModule	AuthController	AuthService	POST /auth/register, POST /auth/login
UsersModule	UsersController	UsersService	GET /users, GET /users/:id, POST /users
DistrictModule	DistrictController	DistrictService	GET/POST/PUT/DELETE /district
InfrastructureTypeModule	InfrastructureTypeController	InfrastructureTypeService	GET/POST/PUT/DELETE /infrastructure-type
InfrastructureObjectModule	InfrastructureObjectController	InfrastructureObjectService	GET/POST/PUT/DELETE /infrastructure-object
EventModule	EventController	EventService	GET/POST/PUT/DELETE /event

В таблице 4 приведены основные эндпоинты REST API, реализованные в приложении.

Таблица 4 - Эндпоинты REST API

Метод	URL	Описание
POST	/auth/register	Регистрация нового пользователя
POST	/auth/login	Авторизация, возврат JWT-токена
GET	/users	Получить список всех пользователей (in-memory)
GET	/users/:id	Получить пользователя по ID
POST	/users	Создать пользователя (in-memory); поля: name, email (обязательные), age (необязательное)
GET	/district	Получить список всех районов
GET	/district/:id	Получить район по ID
POST	/district	Создать район
PUT	/district/:id	Обновить район

Метод	URL	Описание
DELETE	/district/:id	Удалить район
GET	/infrastructure-type	Список типов инфраструктуры
POST	/infrastructure-type	Создать тип инфраструктуры
PUT	/infrastructure-type/:id	Обновить тип инфраструктуры
DELETE	/infrastructure-type/:id	Удалить тип инфраструктуры
GET	/infrastructure-object	Список объектов инфраструктуры
POST	/infrastructure-object	Создать объект инфраструктуры
PUT	/infrastructure-object/:id	Обновить объект инфраструктуры
DELETE	/infrastructure-object/:id	Удалить объект инфраструктуры
GET	/event	Список событий
POST	/event	Создать событие
PUT	/event/:id	Обновить событие
DELETE	/event/:id	Удалить событие

Пример реализации метода получения списка районов в сервисе:

```
findAll(): Promise<District[]> {
  return this.districtRepo.find({
    order: { name: 'ASC' },
  });
}
```

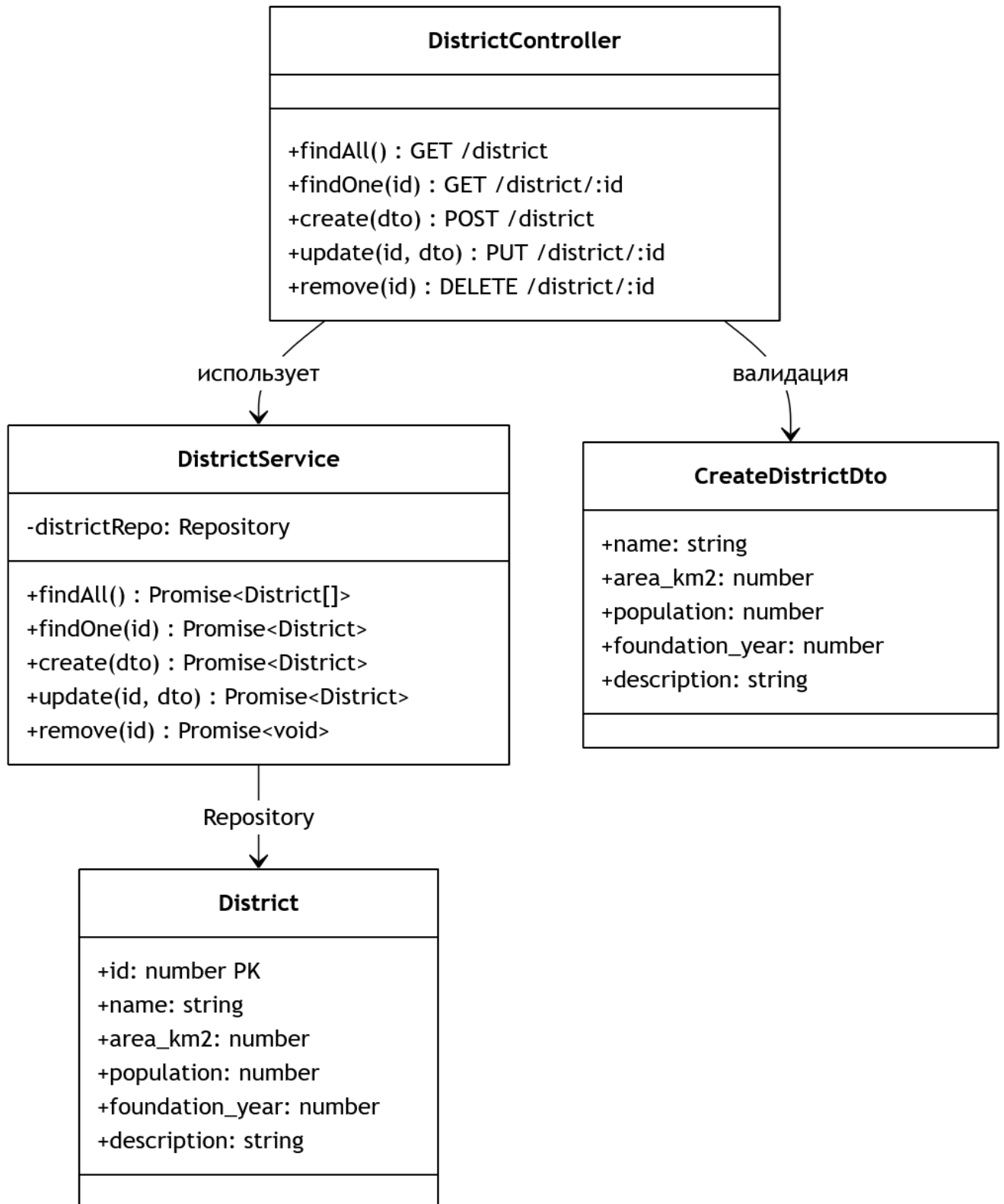


Рисунок 2 – Диаграмма классов модуля District

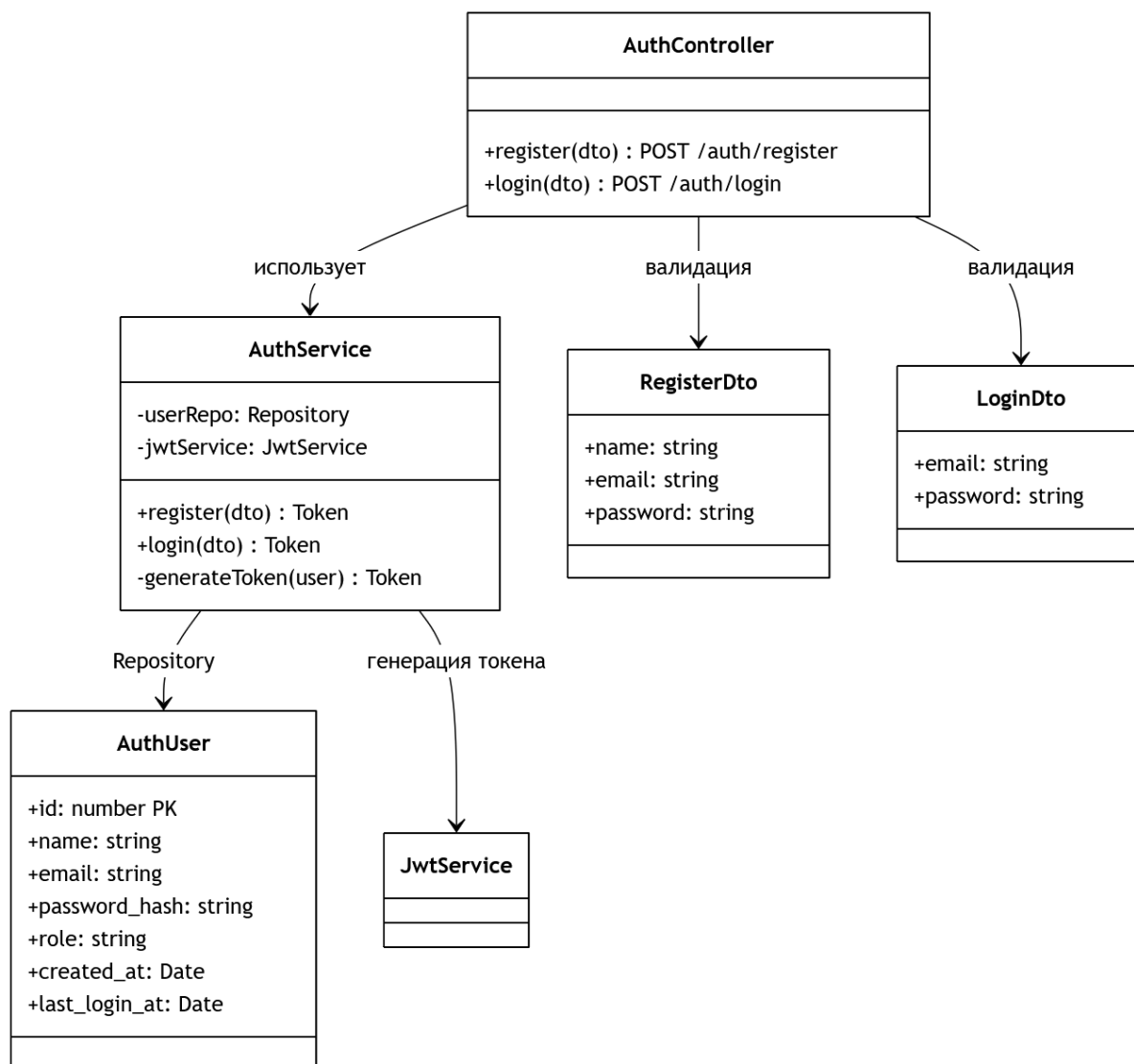


Рисунок 3 – Диаграмма классов модуля Auth

6. Описание клиентской части

Клиентская часть приложения разработана с использованием библиотеки React 18 [3] и инструмента сборки Vite. Навигация между разделами реализована с помощью React Router DOM. Для HTTP-запросов к серверу используется Axios. Глобальное состояние аутентификации управляется через React Context API (AuthContext), что позволяет любому компоненту получить доступ к данным текущего пользователя и его роли. Пользовательский интерфейс выполнен в минималистичном стиле на чистом HTML5 и CSS3 (без внешних UI-библиотек) в холодной цветовой гамме: светлый фон #f8fafc, акцентный цвет #0ea5e9. Адаптивная сетка карточек реализована через CSS Grid.

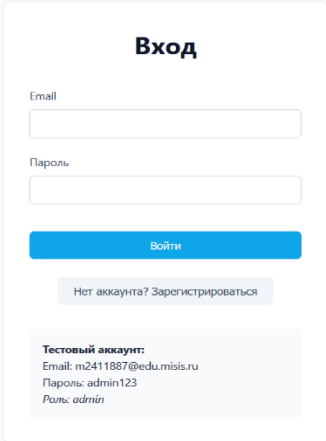
Для защиты маршрутов на клиентской стороне реализован компонент ProtectedRoute, который перенаправляет неаутентифицированного пользователя на страницу /login. Кнопки создания, редактирования и удаления записей отображаются

условно - только для пользователей с ролью «admin». В навигационной панели отображается имя и роль текущего пользователя.

Интерфейс приложения состоит из следующих основных страниц, доступных через навигационную панель:

- Страница входа и регистрации (/login) - форма аутентификации с переключением режимов «Вход» и «Регистрация». После успешного входа JWT-токен и данные пользователя сохраняются в LocalStorage, происходит перенаправление на главную страницу;
- Районы (/) - главная страница с карточками районов, строкой поиска по названию и описанию, кнопкой «+ Добавить» для admin;
- Объекты инфраструктуры (/objects) - табличное представление объектов с колонками: номер, название, адрес, район, тип, год постройки, действия;
- События (/events) - карточки событий с указанием даты и района проведения, поиском по названию;
- Типы инфраструктуры (/types) - таблица справочника типов объектов.

На каждой странице реализованы следующие элементы интерфейса: строка поиска или фильтрации; кнопка добавления новой записи (только для admin); таблица или карточки с данными; кнопки «Изменить» и «Удалить» для каждой записи (только для admin); модальное окно с формой для добавления и редактирования данных; подтверждение удаления через window.confirm.



Вход

Email

Пароль

Войти

[Нет аккаунта? Зарегистрироваться](#)

Тестовый аккаунт:
Email: m2411887@edu.misis.ru
Пароль: admin123
Роль: admin

Рисунок 4 - Страница входа в систему

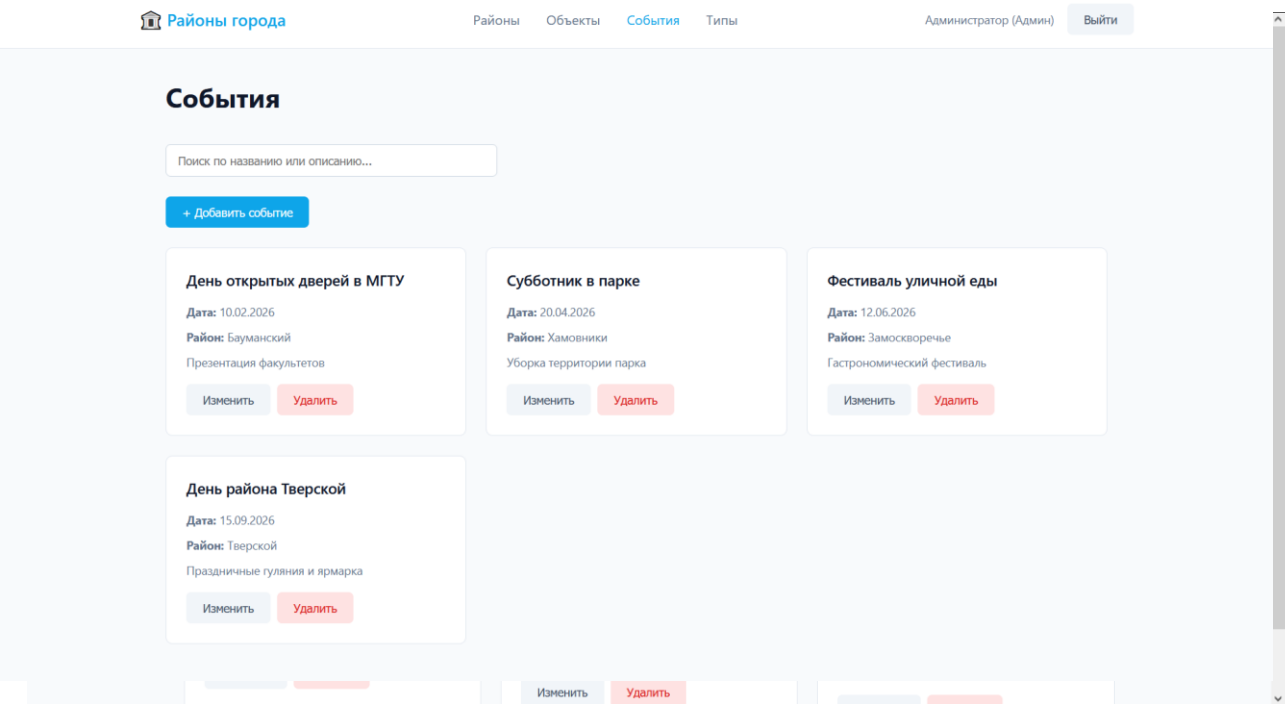


Рисунок 5 - Главная страница «Районы»

Страница «Районы» отображает карточки с информацией о районе: название, площадь, численность населения, год основания, описание. Для пользователей с ролью «admin» отображаются кнопки «Изменить» и «Удалить».

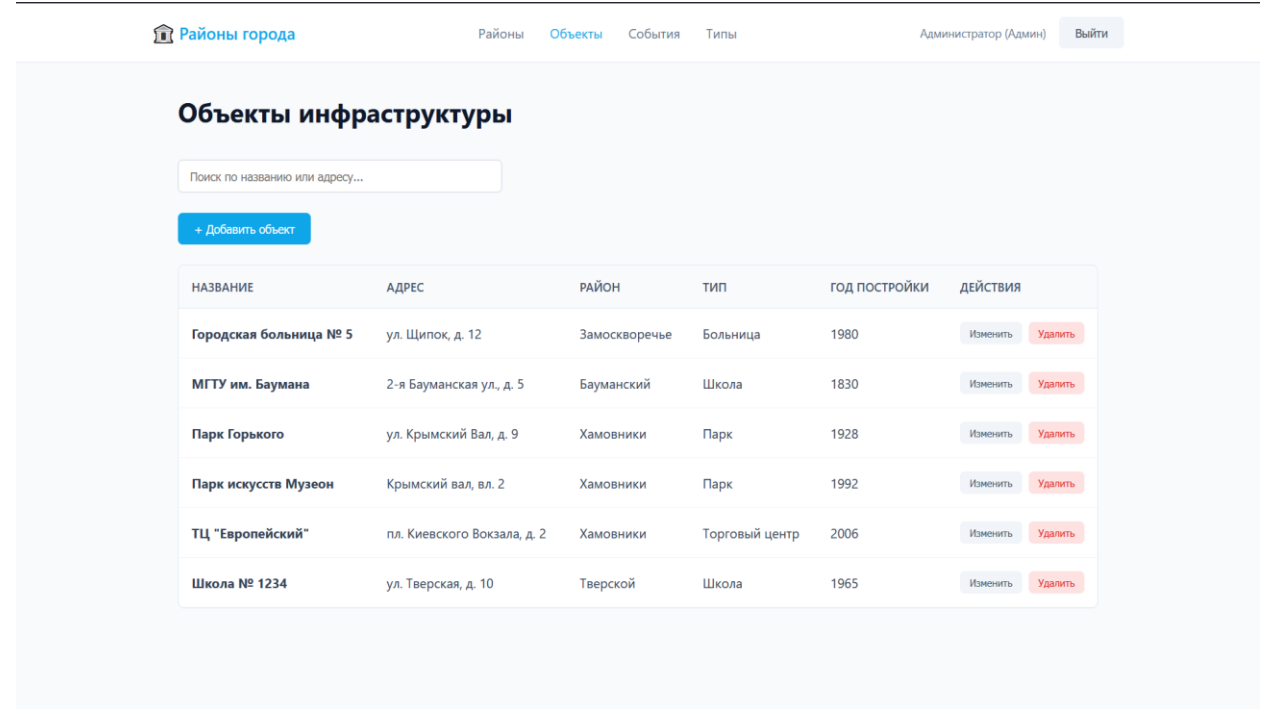


Рисунок 6 - Страница «Объекты инфраструктуры»

Рисунок 7 - Страница «События»

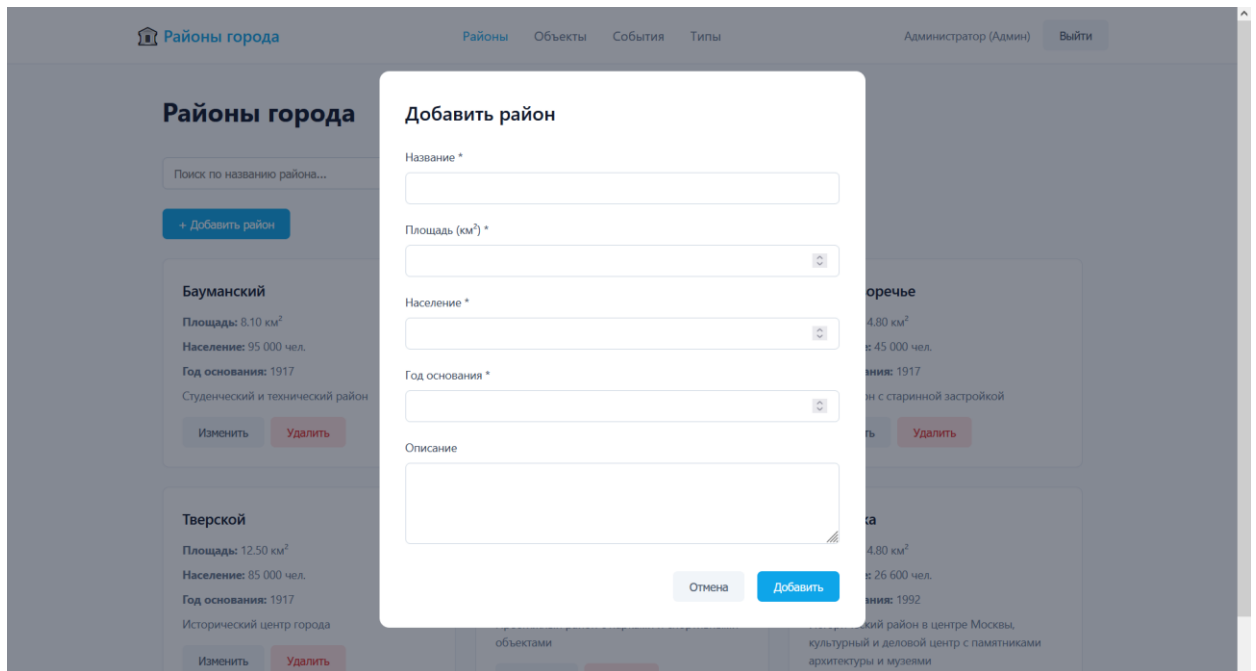


Рисунок 8 - Модальное окно добавления района

[illegible]

Рисунок 9 - Просмотр таблицы district в DBeaver

	 123 id	AZ name	AZ email	AZ password_hash	AZ role	 created_at	 last_login_at
1	2	Иван	ivan@mail.ru	\$2b\$10\$hg/R7NgBo1ucHy9szRPve91Z5kSwwjBgD2okrPw5plhmPSFmOSSS	viewer	2026-06-08 06:57:29.747	2026-06-08 06:57:58.41
2	1	Администратор	m2411887@edu.misis.ru	\$2b\$10\$hdeUUMUuU7/1zfDJ/VZEOTIHFshz2Cozg9N64aCfoW.M6pSMN/aie	admin	2026-06-08 06:44:44.366	2026-06-08 09:12:20.51

Рисунок 10 – Просмотр таблицы auth_user в DBeaver

7. Заключение

В рамках данной курсовой работы была разработана предметная область «Район», посвящённая систематизации информации о городских районах, объектах их инфраструктуры и проводимых событиях. Основная проблема - отсутствие единого удобного инструмента для структурированного хранения и управления данными о районах

города - была решена путём создания полнофункционального клиент-серверного веб-приложения с системой JWT-аутентификации и разграничением прав доступа по ролям.

В процессе выполнения работы были использованы следующие технологии и инструменты: фреймворк NestJS для разработки серверной части с REST API; библиотека React для создания пользовательского интерфейса; СУБД PostgreSQL для хранения реляционных данных; ORM TypeORM для взаимодействия с базой данных; инструмент сборки Vite; HTTP-клиент Axios; библиотеки @nestjs/jwt и bcryptjs для реализации аутентификации; class-validator и class-transformer для валидации данных; система контроля версий Git; инструмент DBeaver для администрирования базы данных.

В результате выполнения работы были получены следующие результаты: разработана реляционная база данных, включающая 5 таблиц (district, infrastructure_type, infrastructure_object, event, district_audit_log), 3 представления, 3 функции, 3 хранимые процедуры и 3 триггера; реализован REST API сервер с 22 эндпоинтами и системой JWT-аутентификации; разработан пользовательский интерфейс с полным CRUD для четырёх сущностей и разграничением прав доступа по ролям admin/viewer; реализован модуль Users с хранением данных в памяти; обеспечено логирование HTTP-запросов. Исходный код проекта размещён в открытом репозитории [6].

Список использованных источников

1. Официальная документация PostgreSQL [Электронный ресурс]. - URL: <https://www.postgresql.org/docs> - Текст: электронный. (дата обращения: 05.06.2026).
2. Официальная документация NestJS [Электронный ресурс]. - URL: <https://docs.nestjs.com> - Текст: электронный. (дата обращения: 03.06.2026).
3. Официальная документация React [Электронный ресурс]. - URL: <https://react.dev> - Текст: электронный. (дата обращения: 02.06.2026).
4. Официальная документация TypeORM [Электронный ресурс]. - URL: <https://typeorm.io> - Текст: электронный. (дата обращения: 02.06.2026).
6. Исходный код проекта [Электронный ресурс]. - URL: https://github.com/fayzeel/BIVT-24-5_Rochev_21_District.git - Текст: электронный. (дата обращения: 08.06.2026).